ARM Cortex-M Mimarisinde Kesme (Interrupt) ve İstisna (Exception) Yönetimi: Derinlemesine Teknik Eğitim Raporu

1. Giriş ve Temel Mantık: Kesme ve İstisna Kavramları

Gömülü sistem tasarımında ve alt seviye (low-level) sistem programlamada, bir CPU (Central Processing Unit - Merkezi İşlem Birimi: Bilgisayar sistemlerinde komutları işleyen ve aritmetik/mantıksal işlemleri yürüten ana işlem birimi) çekirdeğinin dış dünyadaki fiziksel olaylara ve içsel donanımsal durumlara zamanında, öngörülebilir ve güvenilir yanıtlar verebilmesi sistem mimarisinin temel taşıdır. Gerçek zamanlı sistemlerde zamanlama gereksinimlerinin karşılanması, program kontrol akışının (control flow) ne zaman ve nasıl yön değiştirileceğinin hassas bir şekilde yönetilmesine bağlıdır. Bu denetim mekanizmasının temelini **İstisna (Exception)** ve **Kesme (Interrupt)** kavramları oluşturur.

Temel Tanımlar ve Kavramsal Altyapı

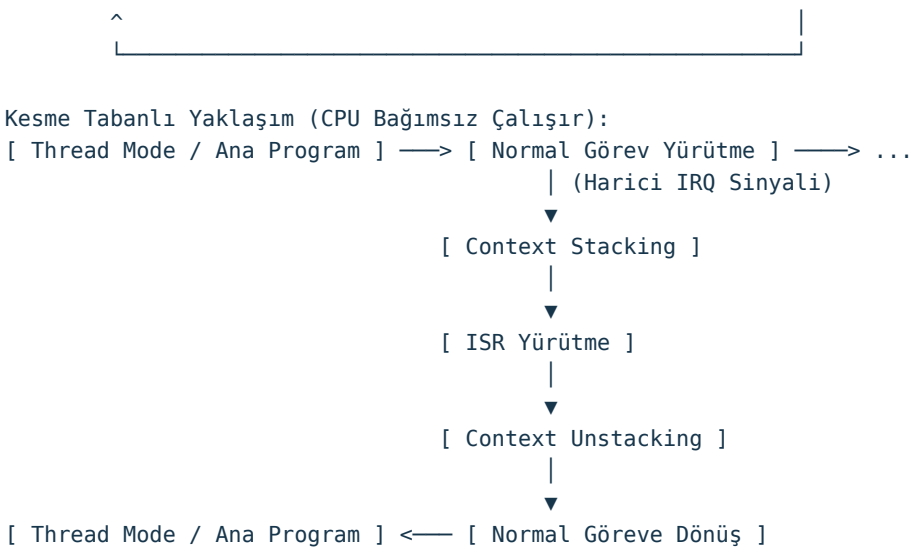
Teknik literatürde ve ARM (Advanced RISC Machines - Gelişmiş RISC Makineleri: Mikrodenetleyici ve işlemci mimarileri tasarlayan yarı iletken firması) mimari standartlarında bu iki kavram şu şekilde tanımlanmaktadır:

- **İstisna (Exception - Mimariden Kaynaklı Çekirdek Durumu):** İşlemci çekirdeğinin iç yapısında veya bellek mimarisinde meydana gelen, komut yürütme aşamasında tespit edilen sistem hataları (örneğin geçersiz bellek erişimi, hiza hatası, tanımsız komut çalıştırma) veya özel yazılım istekleri (sistem çağrıları) sonucunda ana program akışının donanımsal olarak durdurulması ve çekirdeğin önceden tanımlanmış özel bir işleyici koda sıçraması durumudur.
- **Kesme / Donanım Kesmesi (Interrupt - Donanım Kesme İsteği):** İşlemci çekirdeğinin tamamen dışında yer alan dâhili veya harici çevre birimlerinden (örneğin Zamanlayıcı / Timer, UART - Universal Asynchronous Receiver-Transmitter - Evrensel Asenkron Alıcı-Verici: Seri veri iletimi sağlayan donanım birimi, GPIO - General-Purpose Input/Output - Genel Amaçlı Giriş/Çıkış: Çevre birimleri ile dijital veri alışverişi yapan mikrodenetleyici pini) gelen ve işlemciye harici bir olayın gerçekleştiğini bildiren elektriksel veya donanımsal uyarı sinyalleridir.

İşlemci Denetim Akışının Kesilme Mekanizması ve Polling Yaklaşımına Üstünlüğü

Gömülü yazılım geliştirmede harici çevre birimlerinin durumunu takip etmek için iki temel mimari yaklaşım kullanılır: **Sorgulama (Polling)** ve **Kesme Tabanlı Sürüm (Interrupt-Driven Execution)**.

```
Sorgulama (Polling) Yaklaşımı (CPU Kilitlenir):
[ CPU Ana Döngüsü ] ──> [ Bayrak Kontrol Et ] ──(Hazır Değil)──┐
```



1. **Sorgulama (Polling) Yaklaşımı:** İşlemci çekirdeği, bir sonsuz döngü içerisinde ilgili çevre biriminin durum yazmacını (status register) sürekli olarak okur. Çevre birimi veriyi hazırlayana veya işlemi tamamlayana kadar CPU bu döngüde bloklanır. Bu durum, işlemci saat döngülerinin (CPU clock cycles) tamamen boşa harcanmasına, sistemin enerji verimliliğinin düşmesine ve daha acil görevlerin zamanında çalıştırılamamasına (zamanlama ihlali / deadline miss) neden olur.
2. **Kesme Tabanlı Yaklaşım:** İşlemci, harici olay gerçekleşene kadar kendi ana program akışını (Thread Mode) kesintisiz olarak yürütür veya ultra düşük güç modunda (Sleep Mode) bekler. Donanımsal kesme sinyali tetiklendiğinde, işlemci mevcut komut adımını tamamlar, yürütmekte olduğu ana programın durumunu (bağlam / context) donanımsal olarak bellek yığınına saklar (stacking) ve ilgili kesmeye atanmış ISR (Interrupt Service Routine - Kesme Hizmet Programı) fonksiyonuna sıçrar. ISR tamamlandığında yığındaki bağlam geri yüklenir (unstacking) ve ana program tam kaldığı noktadan yürütülmeye devam eder.

Kesme tabanlı yaklaşımın deterministik (belirlenebilir) ve gerçek zamanlı sistemlerde sunduğu temel avantajlar şunlardır:

- **Yüksek İşlemci Verimliliği:** CPU, çevre birimlerinin yavaş süreçlerini beklemek yerine sadece veri hazır olduğunda işleme katılır.
- **Düşük ve Deterministik Tepki Süresi:** Harici bir olay meydana geldiğinde işlemci donanımsal gecikme süresi içerisinde derhal ilgili koda yönlenir.
- **Enerji Optimizasyonu:** İşlemci, olay gelene kadar WFI (Wait For Interrupt - Kesme Bekle: İşlemciyi bir kesme gelene kadar düşük güç moduna sokan çekirdek komutu) veya WFE (Wait For Event - Olay Bekle: İşlemciyi donanımsal bir olay sinyali gelene kadar düşük güç moduna sokan çekirdek komutu) komutlarıyla derin uyku modlarında tutulabilir.

Dâhili Çekirdek İstisnaları ve Harici Çevre Birimi Kesmelerinin Ayrımı

ARM Cortex-M mimarisinde işlemciye gelen tüm yönlendirme talepleri geniş anlamda birer "İstisna" olarak kabul edilir. Ancak kaynağına göre sistem mimarisinde iki gruba ayrılırlar:

1. **Dâhili Çekirdek İstisnaları (Internal Core Exceptions):** Doğrudan ARM çekirdeğinin içerisindeki kontrol birimleri, bellek yönetim birimi veya hata tespit blokları tarafından

üretilir. Sistem sıfırlaması (**Reset**), maskelenemeyen kesme (**NMI**), donanımsal hata (**HardFault**), bellek yönetim hatası (**MemManage**), veri yolu hatası (**BusFault**), kullanım hatası (**UsageFault**), sistem çağrıları (**SVC**, **PendSV**) ve dâhili zamanlayıcı (**SysTick**) bu gruptadır. Vektör tablosunun ilk 15 elemanını oluştururlar.

2. **Harici Çevre Birimi Kesmeleri (External Peripheral Interrupts):** Silicon Vendor (çip üreticisi; örneğin STMicroelectronics, NXP, Texas Instruments, Renesas) tarafından ARM çekirdeğinin etrafına eklenen çevre birimlerinden (UART, SPI, I2C, Timers, ADC, GPIO vb.) gelen kesmelerdir. Bunlar **IRQ** (Interrupt Request) kanalları olarak adlandırılır ve NVIC (Nested Vectored Interrupt Controller - Yuvalanmış Vektörlü Kesme Denetleyicisi) birimi tarafından yönetilirler.

Öncelik Sayısı ve İşlevsel Aciliyet Kavramsal Ayrımı

Gömülü sistem mühendisliğinde en çok kafa karışıklığına yol açan hususlardan biri "Öncelik" kelimesinin kullanımıdır. Rapor boyunca anlam karmaşasını önlemek için şu iki tanım kesin hatlarla ayrılmıştır:

- **Sayısal Öncelik Değeri (Priority Number):** NVIC yazmaçlarına yazılan veya CMSIS (Cortex Microcontroller Software Interface Standard - Cortex Mikrodenetleyici Yazılım Arayüzü Standardı) fonksiyonlarına iletilen tamsayı değeridir (**0, 1, 2, …**).
- **İşlevsel Aciliyet (Urgency):** Bir kesmenin diğer çalışan kodları yarıda kesebilme (preemption) gücüdür. ARM Cortex-M mimarisinde sayısal öncelik değeri **küçüldükçe**, işlevsel aciliyet **artar**. Yani 0 sayısal değeri en yüksek aciliyeti temsil eder.

Temel Temel Terimler ve Kavramlar Listesi

Gömülü sistem mimarisinde kullanılan temel kavramların net tanımları aşağıdadır:

- **IRQ (Interrupt Request - Kesme İsteği):** Donanım çevre birimlerinin işlemciye bir hizmet talebi olduğunu bildirmek amacıyla gönderdiği elektriksel sinyal veya mantıksal durum.
- **ISR (Interrupt Service Routine - Kesme Hizmet Programı):** Belirli bir kesme veya istisna oluştuğunda çalıştırılmak üzere belleğe yüklenmiş, doğrudan ilgili kesme vektörüne bağlı özel C/C++ fonksiyonu.
- **NVIC (Nested Vectored Interrupt Controller - Yuvalanmış Vektörlü Kesme Denetleyicisi):** ARM Cortex-M çekirdeğine entegre edilmiş, kesmelerin öncelik sırasını, yuvalanmasını ve vektör adreslerini donanımsal olarak yöneten merkezi denetleyici.
- **Vector Table (Vektör Tablosu):** Kesme veya istisna oluştuğunda çalıştırılacak olan ISR fonksiyonlarının başlangıç adreslerini (işaretçilerini) sırasıyla tutan, bellek haritasında özel bir adrese yerleştirilmiş tablo.
- **Pending State (Bekleme Durumu):** Bir kesme isteğinin donanım tarafından algılandığı ancak daha yüksek aciliyete sahip başka bir işlem yürütüldüğü için henüz CPU tarafından hizmet verilmediği durum.
- **Active State (Aktif Durum):** Bir kesme isteğinin CPU tarafından kabul edildiği ve ilgili ISR fonksiyonunun şu anda yürütülmekte olduğu durum.
- **Enable/Disable Bit (Etkinleştirme/Devre Dışı Bırakma Biti):** Belirli bir IRQ hattının veya çevre birimi kesmesinin NVIC seviyesinde işleme alınıp alınmayacağını kontrol eden

mantıksal yazmaç biti.

- **Masking (Maskeleme):** Kesme isteklerinin geçici veya kalıcı olarak CPU çekirdeğine ulaşmasının yazılımsal olarak (örneğin **PRIMASK** veya **BASEPRI** yazmaçları ile) engellenmesi işlemi.
- **Context/Stacking (Bağlam / Yığına Saklama):** Kesme oluştuğu anda ana programın çalışma durumunu gösteren işlemci yazmaçlarının (**R0-R3**, **R12**, **LR**, **PC**, **xPSR**) otomatik olarak bellek yığınına (Stack) itilmesi süreci.
- **Interrupt Return (Kesmeden Dönüş):** ISR fonksiyonunun tamamlanmasının ardından, yığına saklanmış olan önceki işlemci bağlamının geri yüklenerek ana program akışına dönülmesi süreci.

---

2. ARM Cortex-M ve NVIC Mimarisi

ARM Cortex-M mimarisinde kesme yönetimi, çekirdeğin ayrılmaz bir parçası olan **NVIC** birimi tarafından tamamen donanımsal seviyede yürütülür. Geleneksel mikrodenetleyici mimarilerinin aksine NVIC, kesme yanıt süresini (interrupt latency) en aza indirmek ve karmaşık yazılım sarmalayıcılarına (software wrappers) olan ihtiyacı ortadan kaldırmak üzere tasarlanmıştır.

NVIC'in Mimarideki Rolü ve Çalışma Mekanizması

Eski nesil işlemci mimarilerinde bir kesme geldiğinde, yazılım ortak bir kesme vektörüne sıçrar ve hangi çevre biriminin kesme ürettiğini belirlemek için yazılımsal sorgulama (polling) yapardı. Bu durum ciddi bir zaman kaybı yaratırdı.

Cortex-M mimarisindeki NVIC ise **Vektörlü (Vectored)** bir mimaridir. Bir kesme sinyali NVIC'e ulaştığında donanım, ilgili kesme numarasına (**IRQn**) karşılık gelen ISR adresini doğrudan Vektör Tablosundan okur ve Program Sayacına (PC - Program Counter: Yürütülecek bir sonraki komutun bellek adresini tutan R15 işlemci yazmacı) yükler.

Ayrıca NVIC **Yuvalanmış (Nested)** bir yapıya sahiptir. Bu yapı sayesinde, daha yüksek aciliyete (daha düşük sayısal öncelik değerine) sahip bir kesme isteği geldiğinde NVIC, o anda yürütülmekte olan daha düşük acillikteki bir ISR'ı yarıda keser (preemption). Düşük acil olan ISR duraklatılır, yeni gelen yüksek acil ISR çalıştırılır ve o bittiğinde duraklatılan ISR kaldığı yerden devam eder.

CMSIS-Core Vektör Tablosu Yapısı ve Mimari Farklılıklar

CMSIS-Core standardına göre Vektör Tablosu bellek haritasının en başında (varsayılan olarak **0x00000000** adresinde) yer alır. Tablonun ilk elemanı (Offset **0x00**), Ana Yığın İşaretçisinin (MSP - Main Stack Pointer: Çekirdek istisnaları ve varsayılan sistem modunda kullanılan bellek yığını işaretçisi) ilk değerini (**__initial_sp**) tutar. İkinci eleman (Offset **0x04**) ise sistem sıfırlandığında çalışacak olan **Reset_Handler** adresidir.

Aşağıdaki tablo, CMSIS-Core v5.6.0 tanımlarına dayalı olarak dâhili çekirdek istisnalarını, ilişkili haberleşme numaralarını (**IRQn**), varsayılan ISR adlarını ve ARM mimari ailelerindeki (Armv6-M, Armv7-M, Armv8-M) bulunabilirlik durumlarını detaylandırmaktadır:

İstisna Vektörü (Exception Vector)	İşleyici Fonksiyon Adı (Handler Function)	IRQn Değeri	Armv6-M (M0/M0+)	Armv7-M (M3/M4)	Armv8-M Baseline	Armv8-M Mainline	Tanım ve Açıklama
NonMaskableInt_IRQn	**NMI_Handler**	-14	Evet	Evet	Evet	Evet	Maskelenemeyen En Yüksek Acil Kesme
HardFault_IRQn	**HardFault_Handler**	-13	Evet	Evet	Evet	Evet	Donanımsal Hata / Sistem Çökme İstisnası
MemoryManagement_IRQn	**MemManage_Handler**	-12	Hayır	Evet	Hayır	Evet	Bellek Koruma Birimi (MPU) Erişim İhlali
BusFault_IRQn	**BusFault_Handler**	-11	Hayır	Evet	Hayır	Evet	Veri / Komut Veri Yolu (Bus) Erişim Hatası
UsageFault_IRQn	**UsageFault_Handler**	-10	Hayır	Evet	Hayır	Evet	Tanımsız Komut / Hizasız Erişim Hatası
SecureFault_IRQn	**SecureFault_Handler**	-9	Hayır	Hayır	Evet	Evet	TrustZone Güvenlik İhlali Hatası
SVCall_IRQn	**SVC_Handler**	-5	Evet	Evet	Evet	Evet	Yazılım Sistem Çağrısı (Supervisor Call)
DebugMonitor_IRQn	**DebugMon_Handler**	-4	Hayır*	Evet	Hayır	Evet	Hata Ayıklama İzleyici İstisnası

İstisna Vektörü (Exception Vector)	İşleyici F onksiyon Adı (Handler Function)	IRQn Değeri	Armv6-M (M0/M0+)	Armv7-M (M3/M4)	Armv8-M Baseline	Armv8-M Mainline	Tanım ve Açıklama
PendSV_I RQn	**PendSV_ Handler**	-2	Evet*	Evet	Evet	Evet	Askıya Alınabilir Sistem Çağrısı (RTOS)
SysTick_I RQn	**SysTick_ Handler**	-1	Evet	Evet	Evet	Evet	Sistem Za manlayıcı Kesmesi

\Not: CMSIS-Core **IRQn_Type** enumerasyon tanımlamalarında **MemoryManagement_IRQn**, **BusFault_IRQn**, **UsageFault_IRQn**, **DebugMonitor_IRQn** ve **PendSV_IRQn** için **[not on Cortex-M0 variants]** (Cortex-M0 varyantlarında bulunmaz) notu düşülmüştür. Armv6-M mimari spesifikasyonunda **PendSV** bulunmakla birlikte, bazı temel Cortex-M0 kütüphane basitleştirmelerinde kısıtlamalar mevcut olabilir.*

IRQn değeri 0 ve daha büyük olan değerler çip üreticisine (vendor) özel harici kesmelerdir (**WWDG_STM_IRQn = 0**, **PVD_STM_IRQn = 1** vb.).

Kesme Kontrolü İçin Fiziksel Yazmaç Mekanizmaları

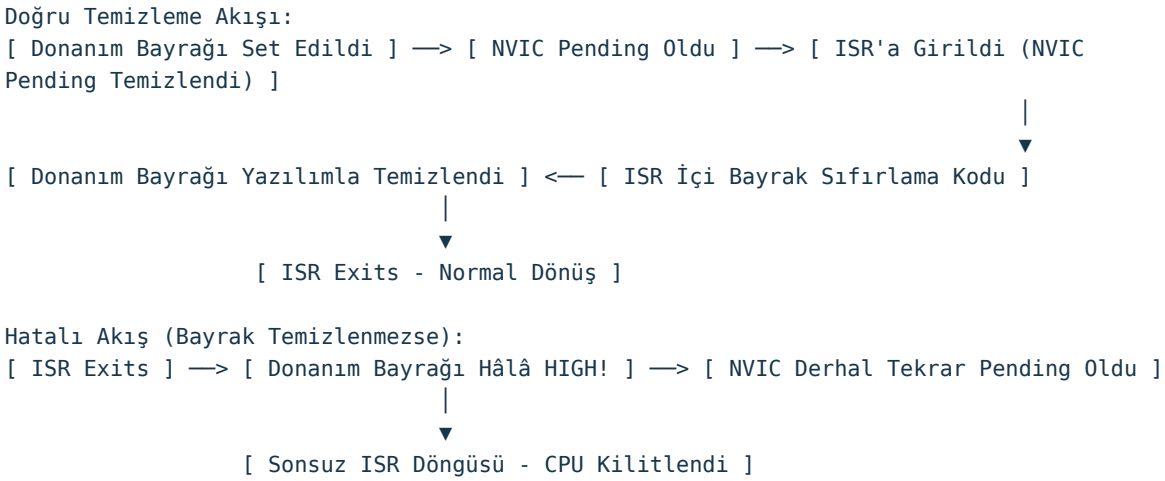
NVIC birimi, kesmeleri yönetmek için özel bellek haritalamalı (Memory-Mapped I/O) yazmaç alanlarına sahiptir:

1. **SETENA (Interrupt Set-Enable Registers) / CLRENA (Interrupt Clear-Enable Registers):**
   - Bir kesmeyi NVIC seviyesinde etkinleştirmek için **SETENA** yazmacının ilgili bitine **1** yazılır.
   - Devre dışı bırakmak için **CLRENA** yazmacının ilgili bitine **1** yazılır. **0** yazmanın hiçbir etkisi yoktur. Bu donanımsal yapı (Write 1 to Set / Clear), oku-değiştir-yaz (Read-Modify-Write) süreçlerinde oluşabilecek çakışmaları (race condition) engeller.
2. **SETPEND (Interrupt Set-Pending Registers) / CLRPEND (Interrupt Clear-Pending Registers):**
   - Bir kesmeyi yazılımsal olarak bekleme (Pending) durumuna geçirmek için **SETPEND** yazmacına **1** yazılır.
   - Bekleyen bir kesme durumunu temizlemek için **CLRPEND** yazmacına **1** yazılır. Kesme aktif hale geldiğinde (ISR başladığında) NVIC bu biti otomatik olarak temizler.
3. **IABR (Interrupt Active Bit Register - Kesme Aktif Bit Yazmacı):**
   - Bir kesmenin o anda CPU tarafından yürütülmekte (Aktif) olup olmadığını gösteren salt okunur (read-only) yazmaçtır. ISR çalıştığı sürece ilgili bit **1** kalır, ISR tamamlanıp kesmeden dönüldüğünde donanım tarafından **0** yapılır.

Periferik Bayrak Temizleme Mantığı ve Kritik Tehlike

Harici bir çevre birimi (örneğin bir Timer veya UART) bir olay gerçekleştirdiğinde, kendi dâhili durum yazmacında bir **Kesme Bayrağı (Interrupt Flag)** kaldırır (**1** yapar). Bu sinyal NVIC'e iletilir ve NVIC ilgili kesmeyi *Pending* durumuna alır.

İşlemci ISR fonksiyonuna girdiğinde, NVIC kendi bünyesindeki *Pending* bitini **otomatik olarak** temizler. **Ancak çevre biriminin kendi içerisindeki kesme bayrağı donanım tarafından otomatik temizlenmez!**



Kritik Mühendislik Uyarısı: Eğer geliştirici ISR fonksiyonu içerisinde çevre biriminin kesme bayrağını yazılımsal olarak temizlemezse (örneğin STM32 veya NXP çevre birimi bayrak sıfırlama yazmacına gerekli yazmayı yapmazsa), çevre biriminden NVIC'e giden elektriksel kesme çizgisi mantıksal **1** seviyesinde kalmaya devam eder. İşlemci ISR sonundaki kesmeden dönüş komutunu çalıştırdığı anda, NVIC sinyali hâlâ yüksek gördüğü için aynı kesmeyi anında tekrar tetikler. Sonuç olarak işlemci sonsuz bir kesme döngüsüne (Infinite ISR Re-entry Loop) girer, ana program (Thread Mode) tamamen durur ve sistem kilitlenir.

---

3. Öncelik Yapılandırması, Sık Yapılan Hatalar ve CMSIS Entegrasyonu

ARM Cortex-M NVIC mimarisinde kesme önceliklerinin yönetimi son derece esnek bir yapı sunar. Ancak tersine dönmüş sayısal mantık ve bit hizalama kuralları nedeniyle mühendislik tasarımlarında en çok hata yapılan alanlardan biridir.

Tersine Öncelik Kuralı (Reversed Priority Rule)

ARM Cortex-M mimarisinde kesme öncelik sayıları ile işlevsel aciliyet arasında ters orantı bulunur:

> Düşük Sayısal Öncelik Değeri = Yüksek İşlevsel Aciliyet (Urgency)

- **Sayısal Öncelik 0:** Yapılandırılabilir en yüksek aciliyete sahip kesme seviyesidir.
- **Sayısal Öncelik N (Örn: 7 veya 255):** Sayısal değer büyüdükçe kesmenin aciliyeti düşer.

Sabit Öncelikler: **NMI** (-14) ve **HardFault** (-13) gibi dâhili çekirdek istisnaları donanımsal olarak sabit negatif değerlere sahiptir ve aciliyetleri yapılandırılabilir tüm harici kesmelerden

yüksektir.

MSB Hizalama Mekanizması (Most Significant Bit Alignment)

ARM mimari spesifikasyonu her kesme için 8 bitlik (1 bayt) bir öncelik yapılandırma alanı tanımlar. Ancak maliyet, alan ve güç optimizasyonu amacıyla mikrodenetleyici üreticileri (Silicon Vendors) bu 8 biti tamamen uygulamak zorunda değildir:

- **Cortex-M0 / Cortex-M0+:** Sabit olarak 2 bit uygulanır (4 farklı aciliyet seviyesi).
- **Cortex-M3 / Cortex-M4:** En az 3 bit (8 seviye) uygulanması zorunludur; üreticiler 3, 4 (16 seviye) veya 5 bit (32 seviye) entegre edebilirler (örneğin Texas Instruments Tiva serisi 3 bit, NXP LPC17xx ve STM32 serisi 4 bit kullanır).

Donanımsal Hizalama Gerçeği: Uygulanan öncelik bitleri, 8 bitlik öncelik baytının **En Önemli Bitlerine (MSB - Most Significant Bit: Bir ikili sayının en yüksek değerlikli, en soldaki biti)** sola dayalı olarak yerleştirilir. Donanımda uygulanmayan alt bitler (LSB) **0** olarak okunur ve yazmaları yok sayılır.

8-bit NVIC Öncelik Baytı Fiziksel Yapısı:

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

2-Bit Uygulaması (Cortex-M0/M0+):

MSB	LSB	0	0	0	0	0	0

Read-Back Mask: 0xC0 (1100 0000b)

3-Bit Uygulaması (Örn: TI Tiva / Stellaris):

MSB	...	LSB	0	0	0	0	0

Read-Back Mask: 0xE0 (1110 0000b)

4-Bit Uygulaması (Örn: NXP LPC17xx / STM32):

MSB	...	...	LSB	0	0	0	0

Read-Back Mask: 0xF0 (1111 0000b)

Eğer donanımın kaç bit desteklediğini tespit etmek için ilgili öncelik yazmacına **0xFF** yazılıp geri okunursa:

- 3-bit destekleyen cihazdan **0xE0** okunur.
- 4-bit destekleyen cihazdan **0xF0** okunur.

Preempt Priority ve Subpriority Ayrımı

Armv7-M (Cortex-M3/M4/M7) mimarisinde, öncelik bitleri iki alt gruba bölünebilir:

1. **Preempt Priority (Kesme Önceliği):** Çalışmakta olan bir ISR'ın daha acil bir kesme tarafından yarıda kesilip kesilemeyeceğini (preemption) belirleyen üst bit grubudur.
2. **Subpriority (Alt Öncelik):** İki veya daha fazla kesme **aynı anda bekleme (pending) durumundayken** hangisinin önce yürütüleceğini belirleyen alt bit grubudur. Alt öncelik,

çalışan bir ISR'ı yarıda kesme gücüne sahip **değildir**.

Bu ayırım, AIRCR (Application Interrupt and Reset Control Register - Uygulama Kesme ve Sıfırlama Kontrol Yazmacı: Sistem sıfırlama ve öncelik gruplamasını kontrol eden SCB yazmacı) içerisindeki **PRIGROUP** (Priority Group - Öncelik Grubu: Öncelik bitlerini Preempt ve Subpriority olarak bölen alan) ile yapılandırılır.

Tavsiye Edilen Mimari Yapılandırma: Sistemde öncelik karmaşasını önlemek ve öngörülebilirliği sağlamak adına tüm bitlerin *Preempt Priority* olarak bırakılması, *Subpriority* bitlerinin sıfırlanması önerilir. Bu durum CMSIS üzerinden **NVIC_SetPriorityGrouping(0U)** çağrısı yapılarak sağlanır.

CMSIS Kaydırma Mantığı ve Miro Samek Tarafından Belirtilen Literatür Hatası

CMSIS kütüphanesi kesme yönetimi için yüksek seviyeli ve donanımdan bağımsız fonksiyonlar sunar:

- **NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)**: Bu fonksiyon parametre olarak kaydırılmamış ham öncelik değerini (örneğin **6**) alır. Fonksiyon, cihazın başlık dosyasında tanımlı **__NVIC_PRIO_BITS** makrosunu kullanarak sola kaydırmayı **içsel olarak** gerçekleştirir:

Fiziksel Yazmaca Yazılan Değer = priority ≪ (8 - __NVIC_PRIO_BITS)

Miro Samek Tarafından Vurgulanan Kritik Hata ve Matematiksel Analiz: Gömülü sistem literatüründe ve hatta bazı tanınmış teknik kitaplarda (örneğin *The Definitive Guide to ARM Cortex-M3*, Section 8.3) **NVIC_SetPriority()** fonksiyonuna parametre aktarılırken manuel olarak sola kaydırılmış bayt değerlerinin gönderildiği görülmektedir.

Örnek Senaryo: 3-bit öncelik destekleyen bir mikrodenetleyicide (**__NVIC_PRIO_BITS = 3**) bir kesmenin aciliyetini **6** (**110b**) yapmak isteyen bir geliştiricinin yanlışlıkla **NVIC_SetPriority(IRQn, 0xC0)** çağırması.

- **0xC0** değeri ikili sistemde **1100 0000b** değerine eşittir (zaten kaydırılmış değer).
- **NVIC_SetPriority()** fonksiyonu bu değeri tekrar sola kaydırır:

İşlem = 0xC0 ≪ (8 - 3) = 0xC0 ≪ 5

0xC0 ≪ 5 = 1100 0000_(b) ≪ 5 = 1 1000 0000 0000_(b) = 0x1800

- Yazmaç bayt boyutunda (8-bit) olduğu için üst bitler kırpılır (truncation):

Yazmaca Yazılan Değer = 0x1800 & 0xFF = 0x00

- **Sonuç:** Aciliyeti 6 yapılması istenen kesme, mantıksal ve matematiksel hata nedeniyle sistemdeki en yüksek aciliyet olan **0** seviyesine ayarlanır!

Buna karşın, **__set_BASEPRI(priority)** gibi doğrudan çekirdek yazmaçlarına erişen CMSIS fonksiyonları **otomatik kaydırma yapmaz**. Yazılımcının kaydırmayı manuel olarak yapması zorunludur:

```
// BASEPRI yazmacına manuel kaydırma yapılarak değer atanması:
__set_BASEPRI(6U << (8 - __NVIC_PRIO_BITS));
```

Kesme Maskeleme Mekanizmaları: PRIMASK ve BASEPRI

İşlemci seviyesinde kesintileri geçici olarak engellemek için iki temel özel yazmaç kullanılır:

1. **PRIMASK (Priority Mask Register - Öncelik Maske Yazmacı):**
   - Komutlar: **CPSID i** (Disable) ve **CPSIE i** (Enable).
   - **PRIMASK** biti **1** yapıldığında, **NMI** ve **HardFault** dışındaki yapılandırılabilir tüm dâhili ve harici kesmeler küresel olarak engellenir. Armv6-M ve Armv7-M mimarilerinin tamamında mevcuttur.
2. **BASEPRI (Base Priority Mask Register - Taban Öncelik Maske Yazmacı):**
   - Yalnızca Armv7-M (Cortex-M3/M4/M7) ve üzeri gelişmiş mimarilerde bulunur.
   - Belirli bir aciliyet eşiğinin altındaki (sayısal öncelik değeri eşit veya daha büyük olan) kesmeleri engellerken, bu eşiğin üstündeki (sayısal değeri daha küçük olan) ultra acil kesmelerin çalışmasına izin verir.
   - **Sıfır Kesme Gecikmesi (Zero Interrupt Latency):** Kritik zamanlamalı motor kontrolü veya güç elektroniği kesmeleri **BASEPRI** maskeleme seviyesinin üstünde tutularak, RTOS kritik bölümlerinde dahi **sıfır gecikme** ile çalıştırılabilir.

---

4. Kesme Gecikmesi (Interrupt Latency), Jitter ve Donanım Optimizasyonları

Gerçek zamanlı gömülü sistemlerde donanımsal zamanlama performansının matematiksel olarak analiz edilmesi şarttır.

Kesme Gecikmesi ve Jitter Tanımları

- **Kesme Gecikmesi (Interrupt Latency):** Harici bir kesme isteğinin (IRQ) donanımsal olarak algılandığı an ile, ilgili ISR fonksiyonunun **ilk komutunun yürütülmek üzere getirildiği (fetch) saat döngüsü** arasında geçen kesin süre (saat döngüsü sayısı).
- **Jitter (Zaman Salınımı):** Kesme gecikme sürelerinin sabitleşmeyip olaydan olaya gösterdiği sapma aralığı. Ses işleme sistemlerinde Jitter sinyal bozulmalarına (distortion), motor kontrol sistemlerinde ise tork dalgalanmalarına ve titreşimlere yol açar.

ARM Cortex-M Çekirdeklerinde Temel Gecikme Tablosu

Sıfır Bekleme Durumlu (Zero Wait-State) bellek sistemlerinde ARM Cortex-M çekirdeklerinin sunduğu donanımsal kesme gecikmeleri aşağıdadır:

Mimari / Çekirdek	Sıfır Bekleme Durumlu Bellek İle Gecikme (Saat Döngüsü / Cycles)
Cortex-M0	16
Cortex-M0+	15
Cortex-M3	12
Cortex-M4	12

Temel Gecikme Hesaplama Varsayımları

Yukarıdaki ideal döngü sayıları şu donanımsal koşulların sağlanması durumunda geçerlidir:

1. Bellek sistemi sıfır bekleme durumundadır (Zero Wait-State) ve otobüs çakışması (bus contention) yaşanmamaktadır.
2. Çevre birimi ile NVIC arasındaki hatta çip seviyesinde ilave köprü/senkronizasyon gecikmesi yoktur.
3. Kesme isteği, o anda çalışan daha yüksek veya eşit aciliyetteki başka bir istisna tarafından engellenmemektedir.
4. Cortex-M4 çekirdeğinde FPU (Floating Point Unit - Yüzer Nokta Birimi: Ondalıklı sayısal hesaplamaları donanımsal olarak yürüten matematiksel birim) aktif ise, varsayılan Tembel Yığına Saklama (Lazy Stacking) özelliği etkindir.
5. Kesme geldiği anda yürütülen komut hizalanmamış bellek erişimi (unaligned transfer) veya bitband erişimi yapmamaktadır (bu durumlar fazladan 1 döngü ekleyebilir).

Otomatik Donanımsal Yığına Saklama (Hardware Stacking) ve EXC_RETURN Mekanizması

ARM Cortex-M mimarisinde ISR fonksiyonları doğrudan standart C fonksiyonları biçiminde yazılır. Derleyiciye özel derleme komutları (**__attribute__((interrupt))**) eklenmesine gerek yoktur. Bunun nedeni, kesme kabul edildiği anda donanımın bağlam saklama (stacking) işlemini otomatik yapmasıdır.

Donanım, kesme girişinde şu 8 yazmacı sırasıyla yığına (Stack) iter:

```
Yığın Bellek (Stack Pointer - SP) Fiziksel Değişimi:
[ Yüksek Bellek Adresi ]
  |  ... Yığın Üstü ...  |
  | ------- xPSR ------- |  <- SP - 32 (Adres Offset 0x1C)
  | -------- PC -------- |  <- SP - 28 (Adres Offset 0x18)
  | -------- LR -------- |  <- SP - 24 (Adres Offset 0x14)
  | ------- R12 -------- |  <- SP - 20 (Adres Offset 0x10)
  | -------- R3 -------- |  <- SP - 16 (Adres Offset 0x0C)
  | -------- R2 -------- |  <- SP - 12 (Adres Offset 0x08)
  | -------- R1 -------- |  <- SP - 8  (Adres Offset 0x04)
  | -------- R0 -------- |  <- SP - 4  (Adres Offset 0x00)
[ Düşük Bellek Adresi ]     <- Yeni SP Değeri
```

Stacking işlemi sırasında **LR** (Link Register - Bağlantı Yazmacı: Fonksiyon veya kesme dönüş adreslerini saklayan R14 işlemci yazmacı) içerisine normal bir geri dönüş adresi yerine **EXC_RETURN** adı verilen özel bir sihirli değer (magic code) yüklenir.

Kritik EXC_RETURN Kodları ve Donanımsal Anlamları:

- **0xFFFFFFF1** (veya **0xFFFFFF01**): Kesmeden çıkışta Handler Mode'a dönüleceğini, ana yığın işaretçisinin (MSP) kullanılacağını belirtir (Yuvalanmış kesme durumu).
- **0xFFFFFFF9** (veya **0xFFFFFF09**): Kesmeden çıkışta Thread Mode'a dönüleceğini ve Ana Yığın İşaretçisinin (MSP) kullanılacağını belirtir.
- **0xFFFFFFFD** (veya **0xFFFFFF0D**): Kesmeden çıkışta Thread Mode'a dönüleceğini ve Süreç Yığın İşaretçisinin (PSP - Process Stack Pointer: RTOS görevleri tarafından kullanılan bağımsız bellek yığını işaretçisi) kullanılacağını belirtir (RTOS görevine dönüş).

ISR tamamlanıp **BX LR** komutu çalıştırıldığında, CPU **LR** içerisindeki **EXC_RETURN** değerini algılar, donanımsal unstacking mekanizmasını başlatır, yığındaki 8 yazmacı geri yükler ve doğru yığın işaretçisini (MSP veya PSP) seçerek ilgili moda geçer.

NVIC Donanımsal Gecikme Optimizasyonları

ARM Cortex-M NVIC birimi, ardışık ve yuvalanmış kesmelerde bellek erişim yükünü ve gecikmeyi en aza indiren üç gelişmiş donanımsal optimizasyona sahiptir:

1. **Tail Chaining (Arka Arkaya Bağlama):** Bir ISR tamamlandığında bekleyen başka bir kesme varsa, donanım yığındaki verileri geri yükleyip (unstacking) hemen ardından yeni kesme için tekrar yığına saklama (stacking) işlemini yapmaz. Unstacking/Stacking süreçlerini tamamen atlayarak doğrudan yeni ISR'a geçer. Bu sayede kesme arası geçiş gecikmesi Cortex-M3/M4'te **6 saat döngüsüne** kadar düşer.
2. **Late Arrival (Geç Varış):** Düşük acilikteki bir kesme geldiğinde donanım yığına saklama (stacking) işlemine başlar. Bu saklama işlemi henüz bitmeden daha yüksek aciliyete sahip başka bir kesme gelirse, işlemci yığına saklama sürecini iptal etmez; ancak saklama bittiğinde ilk olarak yeni gelen yüksek acilikteki ISR'ı çalıştırır.
3. **Pop Pre-emption (Geri Yüklemeyi Yarıda Kesme):** Bir ISR tamamlanıp yığın geri yüklenirken (unstacking) yeni bir kesme gelirse, geri yükleme işlemi derhal durdurulur ve doğrudan yeni ISR fonksiyonuna sıçranır.

Cortex-M0/M0+ İçin Donanımsal Sıfır-Jitter (Zero-Jitter) Özelliği

Joseph Yiu tarafından belirtildiği üzere, bus bekleme durumları (wait-states) veya bellek çakışmaları nedeniyle kesme gecikmesinde küçük zaman salınımları (jitter) oluşabilir. Cortex-M0 ve Cortex-M0+ çekirdeklerinde opsiyonel bir donanım ayarı bulunur. Bu özellik etkinleştirildiğinde, kesme yanıt süresi **sabit olarak en kötü durum gecikmesine (worst-case latency)** zorlanır. Böylece donanım seviyesinde %100 deterministik ve **sıfır jitter** içeren kesme yanıtı elde edilir.

Gelişmiş Güç ve Yanıt Özellikleri

- **Sleep-on-Exit:** Etkinleştirildiğinde, işlemci en son ISR tamamlandığında Thread Mode'a dönmeden doğrudan donanımsal uyku moduna geçer. Kesme güdümlü çalışan sistemlerde ana program döngüsündeki boşa saat harcamalarını ortadan kaldırır (Cortex-M0 uyanma süresi 11 döngüdür).
- **WFI (Wait For Interrupt):** İşlemciyi kesme gelene kadar durdurur. Kesme geldiğinde ISR çalışır ve ardından WFI komutunun altından devam eder.
- **WFE (Wait For Event) ve SEV-ON-PEND:** WFE ile uykuya geçen işlemci, kesme vektörüne girmeden (ISR çalıştırma yükü olmadan) sadece donanımsal bir olay sinyali veya kesme bekleme durumu (**SEV-ON-PEND** aktifse) ile **4 saat döngüsü** kadar kısa bir sürede uyanıp çalışmasına kaldığı yerden devam edebilir.

---

5. Veri Aktarım Yöntemleri: Polling, Interrupt ve DMA Karşılaştırması

Gömülü sistem mimarisinde çevre birimleri ile bellek arasındaki veri akışını yönetmek için üç ana teknik kullanılır:

Mimari Karşılaştırma Tablosu

Kriter	Sorgulama (Polling)	Kesme Tabanlı (Interrupt-Driven)	Doğrudan Bellek Erişimi (DMA)
CPU Kullanımı (CPU Utilization)	Çok Yüksek (%100 kilitlenme)	Orta / Düşük (Sadece olay anında)	İhmal Edilebilir (İşlemciden bağımsız)
Yanıt Süresi / Gecikme	Belirsiz (Döngü süresine bağlı)	Düşük ve Belirli (Deterministic)	Ultra Düşük (Donanımsal tetikleme)
Uygulama Karmaşıklığı	Çok Düşük	Orta (ISR ve Öncelik yönetimi)	Yüksek (Kanal, Adres ve FIFO yapılandırması)
Donanım Kaynağı Gereksinimi	Yok	NVIC birimi	DMA Denetleyicisi birimi (Direct Memory Access)
İdeal Kullanım Senaryosu	Çok basit, zaman kritikliği olmayan sistemler	Seyrek gelen küçük boyutlu veriler (Buton, Komut)	Yüksek hızlı toplu veri aktarımı (ADC, Display, SPI/UART Stream)

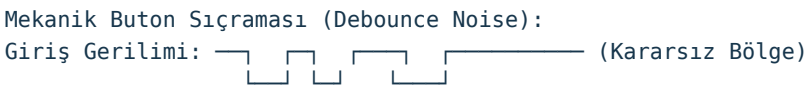
Kısa ISR İlkesi ve Ertelenmiş İş Yükü (Deferred Work)

Kesme Hizmet Programlarının (ISR) en temel tasarım kuralı **mümkün olduğunca kısa tutulmalarıdır**. ISR içerisinde uzun döngüler, karmaşık matematiksel hesaplamalar, **printf** gibi ağır kütüphane çağrıları veya engelleyici (blocking) gecikme fonksiyonları (**delay_ms**) çalıştırmak, sistemdeki diğer kesmelerin gecikmesine (latency spike) ve sistemin gerçek zamanlı garantilerini kaybetmesine yol açar.

Ertelenmiş İş Yükü Mekanizması: ISR içerisinde sadece kritik donanım bayrağı temizlenir, veri hızlıca bir arabelleğe (buffer) alınır ve ham veri işleme görevi ana programa veya bir RTOS görevine (Task) devredilir.

Pratik Senaryo: Buton Arkı Önleme (Debouncing) ve Kesme Fırtınası

Mekanik anahtarlar ve butonlar basıldığı anda mikro saniyeler seviyesinde gürültülü elektriksel sıçramalar (bounce/ark) üretir.




```
Yanlış Yaklaşım: GPIO Kesmesi Doğrudan Bağlanırsa -> "Kesme Fırtınası" (Interrupt Storm)
Gelen ISR Sayısı: 10-50 Adet Arka Arkaya Tetiklenme -> CPU Kilitlenmesi!

Doğru Yaklaşım: Kesme İçerisinde Zamanlayıcı Ertelenmesi (Deferred Software Timer)
GPIO Kesmesi Gelir -> EXTI Kesmesi Kapatılır -> Donanımsal Timer Başlatılır (20ms) ->
ISR'dan Çıkılır
```

```
20ms Sonra Timer ISR Çalışır -> Buton Durumu Kararlı Olarak Okunur -> EXTI Kesmesi
Yeniden Etkinleştirilir.
```

Eğer bir butonun GPIO pinine doğrudan dış kesme (EXTI) atanırsa, tek bir basımda onlarca kesme üretilir. Bu durum **Kesme Fırtınasına (Interrupt Storm)** neden olarak CPU'yu kilitler. Doğru tasarımda kesme alındığı anda dış kesme hattı devre dışı bırakılır, bir zamanlayıcı (Timer) başlatılarak kontrol 20 milisaniye sonraya ertelenir ve buton durumu kararlı halde okunur.

---

6. RTOS İçerisinde Kesme Yönetimi ve Eşzamanlılık (Concurrency)

Gerçek Zamanlı İşletim Sistemlerinde (RTOS - Real-Time Operating System: Kesin zamanlama kısıtlarına sahip gömülü işletim sistemi) kesme yönetimi, çekirdek zamanlayıcısının (Scheduler) bütünlüğünü korumak açısından strict (sıkı) kurallara tabidir.

RTOS FromISR API Kuralları ve Mekanizması

FreeRTOS gibi işletim sistemlerinde, bir ISR içerisinden RTOS nesnelerine (Kuyruk / Queue, Semafor / Semaphore, Event Group) erişileceği zaman **kesinlikle standart API fonksiyonları kullanılamaz**. Yalnızca sonu **FromISR** ile biten özel API fonksiyonları çağrılmalıdır:

- Standart API: **xQueueSend()**, **xSemaphoreGive()**, **vTaskNotifyGive()**
- ISR Uyumlu API: **xQueueSendFromISR()**, **xSemaphoreGiveFromISR()**, **vTaskNotifyGiveFromISR()**

Neden FromISR Kullanılmalıdır?

1. **Engelleme (Blocking) Yasağı:** Standart API fonksiyonları kuyruk doluysa görevi askıya alma (**block**) potansiyeline sahiptir. Ancak ISR bir RTOS görevi değildir; askıya alınamaz veya uyutulamaz.
2. **Bağlam Değişimi (Context Switch): FromISR** fonksiyonları **pxHigherPriorityTaskWoken** adında bir gösterge parametresi alır. Eğer yapılan işlem, o anda çalışan görevden daha yüksek öncelikli bir RTOS görevini uyandırdıysa, ISR çıkışında derhal bağlam değişimi yapılması için bu değişken **pdTRUE** yapılır ve ISR sonunda **portYIELD_FROM_ISR()** çağrılarak geçiş sağlanır.

Görev-ISR Senkronizasyon Araçları: Mutex vs Semafor vs Görev Bildirimleri

- **Mutex (Karşılıklı Dışlama):** Mutex mekanizması **Öncelik Mirası (Priority Inheritance)** algoritmasını kullanır. Öncelik mirası, Mutex'i elinde tutan düşük öncelikli görevin önceliğini geçici olarak yükseltmeyi gerektirir. ISR fonksiyonlarının bir RTOS görevi ve önceliği olmadığı için **Mutex'ler ISR içerisinde ASLA kullanılamaz!**
- **İkili Semafor (Binary Semaphore):** ISR'dan bir göreve olay bildirimi yapmak için uygundur. ISR semaforu verir (**Give**), görev semaforu alır (**Take**).
- **Görev Bildirimleri (Direct-to-Task Notifications):** RAM belleği ve işlemci döngüsü açısından en verimli senkronizasyon yöntemidir. Aracı bir semafor nesnesine ihtiyaç

duyulmadan doğrudan ilgili görevin bildirim değerini günceller.

Kritik Bölümler (Critical Sections) ve BASEPRI Etkileşimi

Kesme emniyetli veri paylaşımı sağlamak için geçici olarak kesmeler kapatılır.

```
// Görev Seviyesi Kritik Bölüm (Thread Mode):
taskENTER_CRITICAL();
/* Bu blokta BASEPRI yazmacı configMAX_SYSCALL_INTERRUPT_PRIORITY seviyesine çekilir.
   Bu seviyenin altındaki kesmeler engellenir. */
g_shared_counter++;
taskEXIT_CRITICAL();

// ISR Seviyesi Kritik Bölüm (Handler Mode):
UBaseType_t uxSavedInterruptStatus;
uxSavedInterruptStatus = taskENTER_CRITICAL_FROM_ISR();
g_shared_counter++;
taskEXIT_CRITICAL_FROM_ISR(uxSavedInterruptStatus);
```

configMAX_SYSCALL_INTERRUPT_PRIORITY değerinden daha yüksek aciliyete (daha küçük sayısal değere) sahip olan kesmeler **BASEPRI** tarafından engellenmez. Bu kesmeler RTOS kritik bölümlerinden etkilenmeyerek **Sıfır Kesme Gecikmesi** elde eder; ancak bu kesmelerin içerisinden RTOS API'leri çağrılamaz.

Eşzamanlılık (Concurrency) Terminolojisi

- **volatile Niteleyicisi:** Derleyiciye, ilgili değişkenin bellekteki değerinin donanım veya başka bir kesme (ISR) tarafından her an değiştirilebileceğini bildirir. Derleyicinin bu değişkeni bir işlemci yazmacına (register) önbelleklemesini (caching) ve değişken erişim optimizasyonlarını engeller. **Tek başına atomiklik veya yarış durumu koruması SAĞLAMAZ.**
- **Atomik İşlemler (Atomic Operations):** Kesintiye uğramadan, tek bir kerede tamamlanan işlemci talimatları (örneğin tek bir belleğe yazma komutu).
- **Yarış Durumu (Race Condition):** İki veya daha fazla kod dizisinin (örneğin Ana program ve ISR) aynı paylaşılan veriye eşzamanlı erişmesi ve erişim sırasına bağlı olarak verinin bozulması durumu.

Gerçek Zamanlı Zaman Bütçesi ve Klasik Çizelgeleme Sınırlamaları

Gömülü sistemlerin analizinde klasik Liu & Layland (Rate Monotonic Scheduling) modeli kullanılır. Ancak bu klasik teorik modeller gerçek dünya donanım dinamiklerini varsayımsal olarak basitleştirir:

- Kesme ve OS Bağlam Değişimi yüklerini (overhead) sıfır kabul ederler.
- Kesilmeyen kritik bölümlerin (non-preemptible critical sections) yol açtığı engelleme sürelerini göz ardı ederler.

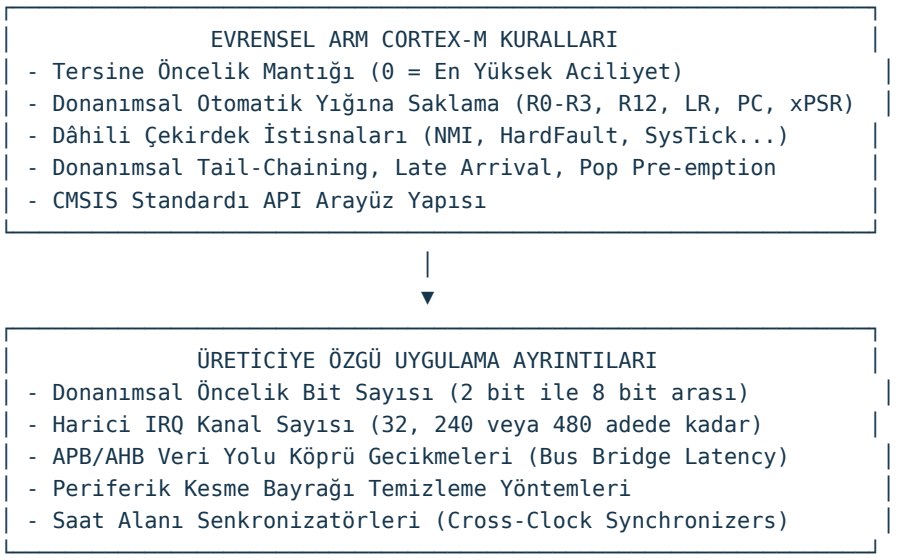
Bu nedenle gerçek zamanlı zaman bütçelemesinde (Time Budgeting) donanımsal kesme süreleri, veri yolu köprü gecikmeleri ve donanım otobüs kilitlenmeleri de hesaplamalara dahil edilmelidir.

---

7. Evrensel Mimari Kurallar ve Cihaz/Üreticiye Özgü Ayrıntıların Ayrımı

ARM Cortex-M mimarisinde kod geliştirirken hangi özelliklerin standart ARM çekirdeğine, hangilerinin ise çip üreticisine (STMicroelectronics, NXP, TI vb.) ait olduğunu bilmek kodun taşınabilirliği açısından kritik önem taşır.

Kategorize Edilmiş Mimari Yapı



Üreticiye Özgü Kritik Donanım Detayları

1. **Saat Alanı Senkronizatörleri (Clock Domain Synchronizers):**

Eğer bir çevre birimi (örneğin GPIO veya Timer) CPU çekirdeğinden farklı bir saat frekansıyla (örneğin daha yavaş bir APB - Advanced Peripheral Bus: Yavaş çevre birimlerini bağlayan bus hattı frekansıyla) çalışıyorsa, çevre biriminden gelen kesme sinyali NVIC'e girmeden önce çift D-Flip-Flop senkronizasyon katmanından geçer. Bu durum gecikmeye fazladan 1-2 saat döngüsü ekleyebilir.

2. **Veri Yolu Köprü Gecikmeleri (Bus Bridge & Write Buffer Latencies):**

İşlemci ISR içerisinde bir GPIO register'ına yazma komutu (**STR**) çalıştırdığında bu işlem 1 döngü sürer. Ancak veri AHB (Advanced High-performance Bus: Çekirdek ve bellek ana bus hattı) köprüsü üzerinden APB hattındaki pusa ulaşıp fiziksel pin durumunu değiştirene kadar 1-2 döngülük ilave veri yolu köprü gecikmesi yaşanır.

---

8. Uygulamalı Sayısal Hesaplama Örnekleri ve Pseudocode

Sayısal Örnek 1: Değişken Baud Hızlarında UART Bayt İletim Süresi ve CPU Yükü Karşılaştırması

Sistem Parametreleri:

- CPU Çalışma Frekansı: **100 MHz** (**1 döngü = 10 ns**)

- Veri Formatı: 8N1 (1 Başlangıç biti, 8 Veri biti, Parite Yok, 1 Bitiş biti = Toplam **10 bit / bayt**)
- ISR İşleme Yükü: Kesme başına 50 CPU saat döngüsü.

1. 9600 Baud Hızında Analiz:

- **1 Bit İletim Süresi (T_(bit)):**

T_(bit) = (1)/(9600 baud) ≈ 104.167 µs

- **1 Bayt (10 Bit) İletim Süresi (T_(bayt)):**

T_(bayt) = 10 × 104.167 µs = 1.04167 ms

- **CPU Yükü ve Polling Kıyaslaması:**
  - *Polling:* CPU 1 bayt beklemek için **1.04167 ms** kilitlenir (**104,167 saat döngüsü** boşa harcanır).
  - *Kesme (Interrupt):* Saniyede yaklaşık 960 bayt gelir. Saniyelik kesme yükü:

Saniyedeki Kesme Sayısı = 960 kesme/sn

Harcanan CPU Döngüsü = 960 × 50 = 48,000 döngü/sn

CPU Yükü (%) = (48,000)/(100,000,000) × 100 = %0.048

2. 115,200 Baud Hızında Analiz:

- **1 Bayt İletim Süresi:**

T_(bayt) = (10)/(115200) ≈ 86.805 µs

Saniyede gelen bayt sayısı: **11,520 bayt/sn**.

- **Kesme Yükü:**

Harcanan CPU Döngüsü = 11,520 × 50 = 576,000 döngü/sn

CPU Yükü (%) = (576,000)/(100,000,000) × 100 = %0.576

3. 921,600 Baud Hızında Analiz:

- **1 Bayt İletim Süresi:**

T_(bayt) = (10)/(921600) ≈ 10.85 µs

Saniyede gelen bayt sayısı: **92,160 bayt/sn**.

- **Kesme Yükü (100 MHz CPU):**

Harcanan CPU Döngüsü = 92,160 × 50 = 4,608,000 döngü/sn

CPU Yükü (%) = (4,608,000)/(100,000,000) × 100 = %4.608

- **Kesme Yükü (Yavaş 16 MHz CPU Üzerinde):**

CPU Yükü (%) = (4,608,000)/(16,000,000) × 100 = %28.8

Mühendislik Değerlendirmesi: Yüksek baud hızlarında (921,600 baud) ve düşük frekanslı işlemcilerde, her **10.85 µs**'de bir kesmeye girmek ciddi bağlam anahtarlama yükü yaratır. Bu seviyede **DMA kullanımı zorunlu hale gelir**.

Sayısal Örnek 2: CMSIS Öncelik Kodlama ve Bit Kaydırma Hesaplaması

Sistem Parametreleri:

- Mikrodenetleyici 4 bit öncelik desteklemektedir (**__NVIC_PRIO_BITS = 4**).
- Ayarlanmak istenen kesme öncelik seviyesi: **6** (**0110b**).

Adım Adım Bit Kaydırma Hesabı:

1. **CMSIS NVIC_SetPriority(IRQn, 6) Fonksiyonu İçsel İşlemi:**

Kaydırma Miktarı = 8 - __NVIC_PRIO_BITS = 8 - 4 = 4 bit

Yazmaca Yazılan Değer = 6 ≪ 4 = 0x06 ≪ 4 = 0x60 (0110 0000_(b))

2. **Direct Register veya __set_BASEPRI Kullanımı:**

Eğer kesmeleri öncelik seviyesi 6 ve altında maskelemek istiyorsak, BASEPRI yazmacına **manuel olarak kaydırılmış değer** atanmalıdır:

```
// Doğru Kullanım:
__set_BASEPRI(6U << (8 - __NVIC_PRIO_BITS)); // BASEPRI içine 0x60 yazılır.

// Hatalı Kullanım:
__set_BASEPRI(6U); // BASEPRI içine 0x06 yazılır, hiçbir kesmeyi engellemez!
```

Uygulamalı Kod/Pseudocode Blokları

A. Doğru Temizleme Mekanizmalı Standart ISR Şablonu

```
#include "LPC17xx.h" // Örnek CMSIS Cihaz Başlığı

// Timer0 Kesme Hizmet Programı (ISR)
void TIMER0_IRQHandler(void) {
    // 1. Durum Kontrolü: Kesme gerçekten Match Channel 0'dan mı geldi?
    if (LPC_TIM0->IR & (1 << 0)) {

        // 2. KRİTİK: Periferik Kesme Bayrağını Temizle
        // LPC17xx mimarisinde IR yazmacına '1' yazmak bayrağı temizler.
        LPC_TIM0->IR |= (1 << 0);

        // 3. Kullanıcı İş Yükü (Mümkün olduğunca kısa tutulmalıdır)
        // Örnek: Tampon adresi güncelleme veya durum kaydı
    }

    // ISR Sonlanması: Donanım otomatik olarak Unstacking yapacaktır.
}
```

B. Kritik Bölüm Koruma: PRIMASK vs BASEPRI Yan Yana Şablonları

```
#include "cmsis_compiler.h"

volatile uint32_t g_sensor_counter = 0; // volatile niteleyicisi zorunludur

// Yaklaşım 1: PRIMASK İle Küresel Kapatma (Armv6-M ve Armv7-M)
void Increment_Counter_PRIMASK(void) {
    uint32_t primask_status = __get_PRIMASK(); // Mevcut durumu kaydet
    __disable_irq(); // CPSID i komutunu çalıştırır (Tüm kesmeler kapalı)

    g_sensor_counter++; // Atomik olmayan Kritik Bölüm

    if (primask_status == 0) {
        __enable_irq(); // CPSIE i komutunu çalıştırır
    }
}

// Yaklaşım 2: BASEPRI İle Seçici Kapatma (Yalnızca Armv7-M / Cortex-M3/M4)
// Ultra yüksek acil kesmeler engellenmez (Sıfır Latency Korunur)
void Increment_Counter_BASEPRI(void) {
    // Priority 6 ve altındaki (sayısal >= 6) kesmeleri maskele
    __set_BASEPRI(6U << (8 - __NVIC_PRIO_BITS));

    g_sensor_counter++; // Kritik Bölüm

    __set_BASEPRI(0U); // Maskelemeyi kaldır
}
```

C. RTOS Ertelenmiş Kesme Yapısı (Deferred Interrupt Processing)

```
#include "FreeRTOS.h"
#include "semphr.h"

SemaphoreHandle_t xBinarySemaphore = NULL;

// Kesme Hizmet Programı (ISR)
void EXTI0_IRQHandler(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // 1. Donanım bayrağını temizle
    EXTI_ClearFlag(EXTI_Line0);

    // 2. Ertelenmiş iş için semafor ver (FromISR varyantı ile!)
    xSemaphoreGiveFromISR(xBinarySemaphore, &xHigherPriorityTaskWoken);

    // 3. Yüksek öncelikli görev uyandıysa ISR çıkışında Bağlam Değişimi yap
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

// Ana RTOS İş Görevi (Worker Task - Thread Mode)
void vWorkerTask(void *pvParameters) {
    for (;;) {
        // Semafor gelene kadar görevi engellenmiş (Blocked) modda tut
        if (xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdTRUE) {
            // Ağır hesaplama ve veri işleme süreçleri burada yürütülür
            Process_Heavy_Data();
        }
    }
}
```

9. Yaygın Hatalar, Hata Ayıklama (Debugging) ve Sorun Giderme (Troubleshooting)

Yaygın Hata Senaryoları ve Adım Adım Çözüm Rehberi

Problem 1: İşlemci ISR Fonksiyonundan Çıkamıyor (Sonsuz Döngü)

- **Kök Neden:** ISR içerisinde çevre biriminin donanımsal kesme bayrağının (Interrupt Flag) temizlenmemesi.
- **Çözüm:** ISR fonksiyonunun içerisine çevre birimi bayrak temizleme komutunu ekleyin.

Problem 2: Kesme Tetiklendiği Anda İşlemci HardFault_Handler Fonksiyonuna Düşüyor

- **Kök Neden:**
  1. Vektör tablosundaki ISR isminin yanlış yazılması (C++ name mangling veya isim uyumsuzluğu).
  2. VTOR (Vector Table Offset Register - Vektör Tablosu Öteleme Yazmacı: Vektör tablosunun başlangıç adresini tutan SCB yazmacı) yazmacının RAM Relocation sonrası yanlış yapılandırılması.
  3. Kesme girişinde yığın belleğin (Stack) dolması (Stack Overflow).
- **Çözüm:** Startup dosyasındaki weak handler ismiyle C kodundaki ISR isminin tam eşleştiğini kontrol edin. Stack boyutunu artırın.

Problem 3: FreeRTOS İçi Kesme Çalıştığında Sistem Çöküyor (Kernel Crash / Assert Failure)

- **Kök Neden:**
  1. ISR içerisinde **FromISR** eki olmayan standart FreeRTOS API'sinin çağrılması.
  2. Kesmenin aciliyet seviyesinin, **configMAX_SYSCALL_INTERRUPT_PRIORITY** değerinden daha yüksek (sayısal olarak daha küçük) ayarlanması.
- **Çözüm:** Kesme önceliğini sayısal olarak **configMAX_SYSCALL_INTERRUPT_PRIORITY** değerine eşit veya daha büyük yapın.

Problem 4: Yuvalanmış Kesme (Nested Interrupt) Çalışmıyor

- **Kök Neden: PRIGROUP** yapılandırmasının yanlış yapılması veya tüm bitlerin Subpriority olarak ayarlanması.
- **Çözüm:** Sistem başlangıcında **NVIC_SetPriorityGrouping(0U)** fonksiyonunu çağırarak tüm bitleri Preempt Priority olarak atayın.

Donanımsal Hata Ayıklama Araçları

1. **SWV (Serial Wire Viewer - Seri Tel İzleyici: Tek pin üzerinden izleme verisi aktaran hata ayıklama arayüzü):**

Single-pin **SWO** hattı üzerinden işlemciyi durdurmadan kesme giriş/çıkış zamanlarını ve istisna geçmişini (Exception Trace) gerçek zamanlı grafik olarak sunar.

2. **ETM (Embedded Trace Macrocell - Gömülü İzleme Makro-Hücresi: Tam komut takibi sağlayan gelişmiş trace donanımı):**

Çok bitli trace veri yolu kullanarak tam komut izlemesi (Full Instruction Trace) sağlar. Hangi kesmenin hangi mikrosaniyede ne kadar süreyle çalıştığını %100 doğrulukla analiz eder.

---

10. Değerlendirme Soruları ve Açıklamalı Yanıtlar

S1: ARM Cortex-M mimarisinde öncelik sayısal değeri 2 olan bir kesme ile öncelik sayısal değeri 5 olan bir kesme aynı anda gelirse hangisi önce çalışır? Neden?

Cevap: Öncelik değeri 2 olan kesme önce çalışır. Çünkü Cortex-M mimarisinde "Tersine Öncelik Kuralı" geçerlidir; küçük sayısal değerler daha yüksek işlevsel aciliyeti (Urgency) temsil eder.

S2: NVIC_SetPriority(UART0_IRQn, 4) fonksiyonu çağrıldığında 3-bit öncelik destekleyen bir MCU'nun NVIC yazmacına hangi hex değeri yazılır?

Cevap: 0x80 yazılır. CMSIS fonksiyonu kaydırmayı içsel yapar: **4 ≪ (8 - 3) = 4 ≪ 5 = 128 = 0x80** (İkili: **1000 0000b**).

S3: Bir ISR içerisinde çevre biriminin kesme bayrağının temizlenmesi unutulursa ne olur?

Cevap: İşlemci ISR'dan çıktığı anda kesme hattı hâlâ aktif olduğu için NVIC tarafından derhal tekrar tetiklenir ve işlemci sonsuz bir ISR döngüsüne girerek ana programı dondurur.

S4: Cortex-M3/M4 mimarisinde kesme girişinde donanım tarafından otomatik olarak yığına saklanan 8 yazmaç hangileridir?

Cevap: xPSR, **PC**, **LR**, **R12**, **R3**, **R2**, **R1** ve **R0** yazmaçlarıdır.

S5: Tail Chaining optimizasyonu nedir ve gecikmeyi kaç saat döngüsüne düşürür?

Cevap: Ardışık iki kesme arasında yığın geri yükleme (Unstacking) ve tekrar saklama (Stacking) adımlarını atlayarak doğrudan diğer ISR'a geçiş yapılmasıdır. Gecikmeyi Cortex-M3/M4'te 6 saat döngüsüne düşürür.

S6: FreeRTOS kullanan bir sistemde Mutex neden bir ISR içerisinde verilemez veya alınamaz?

Cevap: Mutex mekanizması "Öncelik Mirası" (Priority Inheritance) içerir. ISR bir RTOS görevi olmadığı için öncelik mirası uygulanamaz ve Mutex engelleme yapamayacağı için çöküşe neden olur.

S7: PRIMASK ile BASEPRI yazmaçları arasındaki temel fark nedir?

Cevap: PRIMASK tüm yapılandırılabilir kesmeleri küresel olarak engeller. **BASEPRI** ise sadece belirli bir öncelik değerinden daha az acil olan kesmeleri engeller, daha acil kesmelerin çalışmasına (Zero Latency) izin verir.

S8: Sleep-on-Exit özelliğinin avantajı nedir?

Cevap: ISR bittiğinde işlemcinin Thread Mode'a dönmeden doğrudan uykuya geçmesini sağlar. Ana program yürütme döngülerini ve yığın yüklerini ortadan kaldırarak güç tasarrufu sağlar.

S9: 9600 baud hızında 8N1 formatındaki bir UART hattından 1 bayt veri gelmesi kaç milisaniye sürer?

Cevap: Toplam 10 bit iletilir. **10 × (1 / 9600) ≈ 1.04167 ms** sürer.

S10: WFE komutunun WFI komutundan temel farkı nedir?

Cevap: WFE, bir olay sinyali veya **SEV-ON-PEND** ile ISR yürütme katmanına girmeden (sadece 4 saat döngüsünde) işlemciyi hızlıca uyandırabilir.

S11: volatile kelimesi bir değişken için neyi garanti eder, neyi garanti etmez?

Cevap: Değişkenin derleyici tarafından yazmaçlara önbelleklenmesini engeller ve her zaman bellekten okunmasını garanti eder. Ancak işlemin atomik olmasını veya yarış durumlarından korunmasını garanti etmez.

S12: Late Arrival donanım optimizasyonu nasıl çalışır?

Cevap: Düşük acilikteki bir kesme için yığına saklama yapılırken daha yüksek acilikte bir kesme gelirse, yığına saklama iptal edilmez ancak tamamlandığında ilk olarak yüksek acil kesmenin ISR'ı çalıştırılır.

---

11. Kapsamlı Kısaltma ve Kavram Sözlüğü (Glossary)

Terim / Kısaltma	Açılımı (İngilizce)	Türkçe Karşılığı	Kapsamlı Açıklama
AHB	Advanced High-performance Bus	Gelişmiş Yüksek Performans Veri Yolu	İşlemci, bellek ve yüksek hızlı çevre birimlerini bağlayan ana sistem bus hattı.
AIRCR	Application Interrupt and Reset Control Register	Uygulama Kesme ve Sıfırlama Kontrol Yazmacı	Öncelik gruplama (**PRIGROUP**) ve sistem sıfırlama ayarlarını tutan SCB yazmacı.
APB	Advanced Peripheral Bus	Gelişmiş Çevre Birimi Veri Yolu	Yavaş çevre birimleri (UART, Timer vb.) için kullanılan köprü bağlantılı veri yolu.
API	Application Programming Interface	Uygulama Programlama Arayüzü	Yazılım bileşenlerinin birbiriyle haberleşmesini sağlayan fonksiyon kütüphanesi.
BASEPRI	Base Priority Mask Register	Taban Öncelik Maske Yazmacı	Belirli bir öncelik seviyesinin altındaki kesmeleri maskeleyen özel işlemci yazmacı.

Terim / Kısaltma	Açılımı (İngilizce)	Türkçe Karşılığı	Kapsamlı Açıklama
CMSIS	Cortex Microcontroller Software Interface Standard	Cortex Mikrodenetleyici Yazılım Arayüzü Standardı	ARM tarafından tanımlanan, tüm Cortex-M işlemciler için

Terim / Kısaltma	Açılımı (İngilizce)	Türkçe Karşılığı	Kapsamlı Açıklama
			evrensel yazılım katmanı.
CPU	Central Processing Unit	Merkezi İşlem Birimi	Komutları işleyen ve mantıksal hesaplamaları yürüten ana işlemci çekirdeği.
DMA	Direct Memory Access	Doğrudan Bellek Erişimi	CPU müdahalesi olmadan çevre birimleri ile bellek arasında veri aktaran donanım birimi.
ETM	Embedded Trace Macrocell	Gömülü İzleme Makro-Hücresi	İşlemcinin tüm komut akışını gerçek zamanlı donanımsal olarak izleyen hata ayıklama birimi.
FPU	Floating Point Unit	Yüzer Nokta Birimi	Ondalıklı sayısal hesaplamaları donanımsal olarak hızlandıran matematiksel birim.
GPIO	General-Purpose Input/Output	Genel Amaçlı Giriş/Çıkış	Dış dünya ile dijital sinyal alışverişi yapan mikrodenetleyici pinleri.
IABR	Interrupt Active Bit Register	Kesme Aktif Bit Yazmacı	Hangi kesmelerin o anda CPU tarafından yürütüldüğünü gösteren NVIC yazmacı.
IRQ	Interrupt Request	Kesme İsteği	Harici çevre birimlerinden NVIC birimine gelen donanımsal kesme talebi.
ISR	Interrupt Service Routine	Kesme Hizmet Programı	Kesme gerçekleştiğinde çalıştırılan özel kesme işleyici fonksiyonu.
LR	Link Register	Bağlantı Yazmacı	Fonksiyon veya kesme dönüş adreslerini saklayan işlemci yazmacı (R14).
MSB	Most Significant Bit	En Önemli Bit	Bir ikili sayının veya baytın en yüksek değerlikli (en soldaki) biti.

Terim / Kısaltma	Açılımı (İngilizce)	Türkçe Karşılığı	Kapsamlı Açıklama
NMI	Non-Maskable Interrupt	Maskelenemeyen Kesme	**PRIMASK** ile engellenemeyen, aciliyeti en yüksek dâhili/harici kesme.
NVIC	Nested Vectored Interrupt Controller	Yuvalanmış Vektörlü Kesme Denetleyicisi	ARM Cortex-M kesme ve öncelik yönetimini yürüten donanım birimi.
PC	Program Counter	Program Sayacı	Yürütülecek bir sonraki komutun bellek adresini tutan işlemci yazmacı (R15).
PENDSV	Pending Supervisor Call	Askıya Alınabilir Sistem Çağrısı	RTOS bağlam değişimlerini güvenle yürütmek için kullanılan dâhili istisna.
PRIGROUP	Priority Group	Öncelik Grubu	Kesme önceliğini Preempt ve Subpriority olarak bölen **AIRCR** yazmaç alanı.
PRIMASK	Priority Mask Register	Öncelik Maske Yazmacı	**NMI** ve **HardFault** hariç tüm kesmeleri küresel kapatan 1 bitlik yazmaç.
PSR / xPSR	Program Status Register	Program Durum Yazmacı	İşlemcinin bayraklarını ve aktif kesme numarasını tutan yazmaç.
RTOS	Real-Time Operating System	Gerçek Zamanlı İşletim Sistemi	Zaman kritik görevlerin çizelgelenmesini sağlayan uzmanlaşmış işletim sistemi.
SEV	Send Event	Olay Gönder Komutu	WFE modundaki işlemcileri uyararak uyandırmak için kullanılan assembly komutu.
SVC	Supervisor Call	Denetleyici Çağrısı	Kullanıcı modundan yetkili çekirdek moduna geçiş sağlayan yazılım istisnası.
SWV	Serial Wire Viewer	Seri Tel İzleyici	Tek pin üzerinden gerçek zamanlı kesme ve veri izlemesi sağlayan hata ayıklama arayüzü.

Terim / Kısaltma	Açılımı (İngilizce)	Türkçe Karşılığı	Kapsamlı Açıklama
SYSTICK	System Tick Timer	Sistem Zamanlayıcı	İşletim sistemlerine zaman adımı sağlayan 24-bit dâhili çekirdek zamanlayıcısı.
UART	Universal Asynchronous Receiver-Transmitter	Evrensel Asenkron Alıcı-Verici	Seri haberleşme sağlayan standart donanım çevre birimi.
VTOR	Vector Table Offset Register	Vektör Tablosu Öteleme Yazmacı	Vektör tablosunun RAM veya Flash içerisindeki başlangıç adresini tutan yazmaç.
WFE	Wait For Event	Olay Bekle Komutu	İşlemciyi bir olay sinyali gelene kadar düşük güç moduna sokan komut.
WFI	Wait For Interrupt	Kesme Bekle Komutu	İşlemciyi bir kesme gelene kadar düşük güç moduna sokan komut.

---

12. Kaynakça (References)

1. **Yiu, Joseph.** *A Beginner's Guide on Interrupt Latency - and Interrupt Latency of the Arm Cortex-M processors*. Arm Community Blogs, 2016.
2. **Samek, Miro.** *Cutting Through the Confusion with Arm Cortex-M Interrupt Priorities*. Arm Community Blogs, 2014.
3. **Arm Ltd.** *CMSIS-Core (Cortex-M) Version 5.6.0 - Interrupts and Exceptions (NVIC)*. Arm Software Documentation, 2022.
4. **FreeRTOS Documentation.** *Running the RTOS on an ARM Cortex-M Core*. Amazon Web Services / FreeRTOS Kernel Guide.